

COM3035 – OPERATING SYSTEMS

Vize ve Final Çalışma Soruları

Önemli Not: Bu dokümanda yer alan soruların tamamı, *Operating System Concepts* (A. Silberschatz) adlı ders kitabı ve ilgili dersin geçmiş yıllarda yapılmış sınavları referans alınarak hazırlanmıştır.

PROCESSES

1. Give a solution to the producer consumer problem with a bounded buffer problem in C or pseudo code. Explain your code with necessary comment lines. Does your solution run correctly in a multi-processor system, why?

2. Draw a Tree showing the child/parent relationship of processes created by the following code fragment.

```
pid = fork();  
if (pid != 0)  
    fork();  
else  
    fork();  
fork();
```

3. Draw a state-transition diagram showing a life cycle of a process including short, medium, and long term scheduling queues. Label the states and transitions.

4. Consider the following C program presentation. What output will be at line A?

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int value = 5;

int main() {
    pid_t pid;
    pid = fork();

    if (pid == 0) {
        value += 15;
        return 0;
    }
    else if (pid > 0) {
        wait(NULL);
        printf("PARENT: value = %d",value); /* LINE A */
        return 0;
    }
}
```

5. Describe the differences among short-term, medium-term, and long-term scheduling.

6. Describe the actions taken by a kernel to context-switch between processes.

7. How many processes are created in the following C code presentation?

```
#include <stdio.h>
#include <unistd.h>

int main() {
    fork();
    fork();
    fork();
    return 0;
}
```

8. Draw the process creation diagram using the `fork()` system call.

9. Explain the circumstances under which the line of code marked `printf("LINE J")` in the following C program will be reached.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;
    pid = fork();

    if (pid < 0) {
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) {
        execlp("/bin/ls", "ls", NULL);
        printf("LINE J");
    }
    else {
        wait(NULL);
        printf("Child Complete");
    }
    return 0;
}
```

10. What are the three types of interrupts handled in kernel mode? Explain why they are handled in kernel mode?

11. Using the C program given below, identify the values of pid values at lines A, B, C, and D (Assume that the actual pid values of the parent and child are 2600 and 2603, respectively).

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid, pid1;
    pid = fork();
    if (pid < 0) {
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) {
        pid1 = getpid();
        printf("child: pid = %d",pid); /* A */
        printf("child: pid1 = %d",pid1); /* B */
    }
    else {
        pid1 = getpid();
        printf("parent: pid = %d",pid); /* C */
        printf("parent: pid1 = %d",pid1); /* D */
        wait(NULL);
    }
    return 0;
}
```

12. What output will be at Line X and Line Y in the following C program representation?

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
#define SIZE 5

int nums[SIZE] = {0,1,2,3,4};

int main() {
    int i;
    pid_t pid;
    pid = fork();
    if (pid == 0) {
        for (i = 0; i < SIZE; i++) {
            nums[i] *= -i;
            printf("CHILD: %d ",nums[i]); /* LINE X */
        }
    }
    else if (pid > 0) {
        wait(NULL);
        for (i = 0; i < SIZE; i++)
            printf("PARENT: %d ",nums[i]); /* LINE Y */
    }
    return 0;
}
```

13. Four integer numbers given as input to the Parent Process must first be printed into the Shared Memory. The sum of these numbers should be obtained with 2 child processes produced from the parent process, and this sum should be accessed from the parent process and printed. Design such a program in C, Java, or pseudocode.

14. Consider the following C program:

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("A ");

    pid_t pid;
    pid = fork();

    if (pid == 0) {
        printf("B ");
        execl("/bin/date", "date", NULL);
        printf("C ");
    }
    else {
        printf("D ");
    }
    printf("E ");

    return 0;
}
```

- (a) What will be the output of the program?
- (b) Will the output always be the same each time the program is executed, why?

15. Write a simple client–server program in Java or C that communicates over TCP sockets. The server should create a socket that listens on port 8080, while the client should connect to the server using `host = 127.0.0.1` and `port = 8080`. Once connected, the client should send the message "Hello Server!", and the server should display the received message on the screen. Discuss the limitations (disadvantages) of this basic implementation.

16. The following pseudocode shows a simple version of the *Producer–Consumer* problem using a bounded buffer.

PRODUCER

```
while (true) {  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

CONSUMER

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
}
```

Identify and explain two problems with this solution.

17. Consider the “*exactly once*” semantic with respect to the RPC mechanism. Does the algorithm for implementing this semantic execute correctly even if the `ACK` message sent back to the client is lost due to a network problem? Describe the sequence of messages, and discuss whether “*exactly once*” is still preserved.

CPU SCHEDULING

1. Suppose that the following processes arrive for execution at the times indicated. Each process will run for the amount of time listed. In answering the questions, use nonpreemptive scheduling, and base all decisions on the information you have at the time the decision must be made.

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P ₁	0.0	8
P ₂	0.4	4
P ₃	1.0	1

- (a) What is the average turnaround time for these processes with the FCFS scheduling algorithm?
 (b) What is the average turnaround time for these processes with the SJF scheduling algorithm?

2. Consider the following set of jobs to be scheduled for execution on a single CPU system:

<u>Job</u>	<u>Arrival</u>	<u>Time</u>	<u>Priority</u>
J ₁	0	1	2
J ₂	2	8	1
J ₃	3	3	3
J ₄	10	4	2

- (a) Draw a Gantt chart showing preemptive SRJF scheduling.
 (b) Draw a Gantt chart showing preemptive PRIORITY scheduling.
 (c) Which scheduling is more suitable for a real time environment, why?

3. Consider the following set of jobs to be scheduled for execution on a single CPU system:

Process	Arrival time	CPU time
P1	0	4
P2	4	6
P3	5	1
P4	7	3
P5	12	5

(a) Draw a Gantt chart showing preemptive SRJF scheduling.

(b) Calculate the average waiting time and average turnaround time.

(c) Now assume that process P_5 used the CPU for (5ms, 3ms, 5ms) in its 3 previous executions.

What will be the next CPU burst prediction for P_5 if the system uses exponential averaging with equal weights for the most recent past and for the earlier CPU usages.

4. Consider the following set of jobs to be scheduled for execution on a single CPU system:

Process	Arrival Time	CPU Burst
P1	0	4
P2	4	6
P3	6	1
P4	8	3

- (a) Draw a Gantt chart showing preemptive SRJF scheduling.
- (b) Calculate the average waiting time and average turnaround time for the preemptive SRJF.
- (c) Assume that the process P_1 used the CPU for (6ms, 4ms) in its 2 previous executions. What will be the next CPU burst prediction for P_1 if the system uses exponential averaging with equal weights for the most recent past and for the earlier CPU usages.
- (d) What is the time complexity $O(n)$ of the preemptive SRJF algorithm applied?

5. Consider the following set of jobs to be scheduled for execution on a single CPU system:

<u>Process</u>	<u>Arrival Time</u>	<u>Execution Time</u>
P_1	0	2
P_2	3	8
P_3	6	1
P_4	8	3

- (a) Draw a Gantt chart showing preemptive SRJF scheduling.
- (b) Calculate the average waiting time and average turnaround time for the preemptive SRJF.
- (c) Now assume that process P_3 used the CPU for (8ms, 4ms, 8ms) in its 3 previous executions. What will be the next CPU burst prediction for P_1 if the system uses exponential averaging with equal weights for the most recent past and for the earlier CPU usages.
- (d) How can you adjust the prediction formula to take more into account the earlier CPU usages, what is the next prediction for P_3 in your adjusted formula for prediction?

6. Consider the following set of jobs to be scheduled for execution on a single CPU system:

Process	Arrival time	CPU time
P1	0	8
P2	6	6
P3	7	1
P4	9	3
P5	14	5

- (a) Draw a Gantt chart showing preemptive SRJF scheduling.
- (b) Round and Robin 5ms timeslot.
- (c) Calculate and show the average waiting time turnaround time of the above algorithms.
- (d) Now for SRJF assume that process P_2 used the CPU for (5ms, 4ms, 11ms) in its 3 previous executions. What will be the next CPU burst prediction for P_2 if the system uses exponential averaging with equal weights for the most recent past and for the earlier CPU usages.

7. Consider the following set of jobs to be scheduled for execution on a single CPU system:

Process	Arrival time	CPU time
P1	0	8
P2	6	6
P3	7	1
P4	9	3
P5	14	5

(a) Draw a Gantt chart showing preemptive SRJF scheduling.

(b) Calculate and show the average waiting time and turnaround time.

(c) Now assume that process P_3 used the CPU for (10ms, 4ms, 11ms) in its 3 previous executions. What will be the next CPU burst prediction for P_3 if the system uses exponential averaging with $\alpha = 0.6$ for most recent past and $\alpha = 0.4$ for the earlier CPU usages.

8. Consider the following set of processes, with the length of the CPU burst given in milliseconds:

Process	Burst Time	Priority
P_1	2	2
P_2	1	1
P_3	8	4
P_4	4	2
P_5	5	3

The processes are assumed to have arrived in the order P_1, P_2, P_3, P_4, P_5 , all at time 0.

- Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, nonpreemptive priority (a larger priority number implies a higher priority), and Round Robin (quantum = 2).
- What is the turnaround time of each process for each of the scheduling algorithms in part (a)?
- What is the waiting time of each process for each of these scheduling algorithms?
- Which of the algorithms results in the minimum average waiting time (over all processes)?

THREADS

1. Design a multithreaded program that calculates fibonacci numbers. Explain your code with comment lines.

2. Design a multithreaded (3 threads) program that calculates the factorial of a number. Explain your code with comment lines.

3. Describe the actions taken by a kernel to context-switch between kernel-level threads.

4. Consider the following code segment:

```
pid_t pid;
pid = fork();

if (pid == 0) {
    fork();
    thread_create(...);
}
fork();
```

- (a) How many unique processes are created?
- (b) How many unique threads are created?

5. Consider the following multithreaded C program uses the POSIX Pthread library:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int counter = 0;

void *worker(void *arg) {
    int i = 0;

    for (i = 0; i < 1000; i++) {
        counter++;
    }

    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("%d", counter);

    return 0;
}
```

- (a) What will be the final output of this program? Does the output always produce the same result? Explain the underlying reason.
- (b) Identify the problem in the code above. Rewrite or modify the code to ensure the program always produces the correct result.

6. The program shown below uses the Pthreads API. What would be the output from the program at LINE C and LINE P?

```
#include <pthread.h>
#include <stdio.h>
#include <types.h>

int value = 0;

void *runner(void *param);

int main(int argc, char *argv[]) {
    pid_t pid;
    pthread_t tid;
    pthread_attr_t attr;
    pid = fork();

    if (pid == 0) {
        pthread_attr_init(&attr);
        pthread_create(&tid, &attr, runner, NULL);
        pthread_join(tid, NULL);
        printf("CHILD: value = %d",value); /* LINE C */
    }
    else if (pid > 0) {
        wait(NULL);
        printf("PARENT: value = %d",value); /* LINE P */
    }
}

void *runner(void *param) {
    value = 5;
    pthread_exit(0);
}
```

PROCESS SYNCHRONIZATION

1. Briefly explain what the three solution criteria of the Critical Section Problem (CSP) are, and what kinds of problems occur in the system if each of these criteria is not satisfied.

2. Design a solution in pseudocode to solve the Critical Section Problem (CSP). Prove that your solution satisfies all three criteria of the CSP.

3. Given the following solution to the Dining Philosophers Problem. Is the problem solved, why?

```
// Semaphore chopstick[5] initialized to 1
do {
    wait(chopstick[i]);
    wait(chopstick[(i + 1) % 5]);

    // Eat

    signal(chopstick[i]);
    signal(chopstick[(i + 1) % 5]);

    // Think
} while (true);
```

4. How does the following program solve requirements of the CSP?

```
while (true) {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = Swap(&lock, &key);
    waiting[i] = false;

    /* Critical Section */

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = false
    else
        waiting[j] = false

    /* Remainder Section */
}
```


5. Analyze the given algorithm proposed to solve the Critical Section Problem (CSP).

```
do {  
    while (turn != j);  
    // Critical section  
    turn = j;  
    // Remainder section  
}  
while (true);
```

- (a) Identify and correct the shortcomings of this algorithm.
- (b) For two processes, show that the three criteria of the CSP are satisfied in your corrected algorithm.
- (c) Write a non-busy waiting lock (Semaphore) implementation in pseudocode for mutual exclusion.

6. The following code representation proposes a solution to the Readers-Writer Problem using semaphores. Consider the shared variables are defined as:

```
semaphore wrt_mutex = 1;
semaphore mutex = 1;
int read_count = 0;

void writer() {
    // Code missing...

    signal(wrt_mutex);
}

void reader() {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(wrt_mutex);
    signal(mutex);

    // Critical Section (Reading...)

    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(wrt_mutex);
    signal(mutex);
}
```

(a) Fill in the missing body of the `writer()` process.

(b) This implementation creates a starvation problem. Explain why this happens based on the synchronization logic and provide an example scenario.

(c) Modify the code to resolve the starvation, ensuring that all three Critical Section Problem solution criterias are met.

7. Consider the following solution to the Producer–Consumer Problem using semaphores. Modify the code representation to solve the *starvation* problem.

```
// Producer
while (true) {
    wait(empty);
    wait(mutex);

    signal(mutex);
    signal(full);
}

// Consumer
while (true) {
    wait(full);
    wait(mutex);

    signal(mutex);
    signal(empty);
}
```

8. Consider the Dining Philosophers Problem with n philosophers. Each philosopher needs two forks (left and right) to eat. The following semaphore-based solution is proposed:

```
do {
    wait(chopstick[i]);
    wait(chopstick[(i + 1) % n]);

    // Eat

    signal(chopstick[i]);
    signal(chopstick[(i + 1) % n]);

    // Think
} while (true);
```

It is known that this solution does not satisfy the *Progress* and *Bounded Waiting* requirements of the Critical Section Problem. As a result, both *deadlock* and *starvation* may occur.

- (a) Modify the above code to eliminate deadlock, but where starvation may still be possible. You may use an additional semaphore (e.g., a mutex) if necessary. Briefly explain why deadlock is prevented and why starvation may still occur.
- (b) Further modify the solution to eliminate both deadlock and starvation. You may introduce additional semaphores (e.g., a room, a queue, or waiter semaphore). Explain how your solution satisfies Progress and Bounded Waiting.

9. Below is a proposed solution to the Readers-Writers Problem where readers are serviced before the writer threads.

```
// Writer
while (true) {
    wait(wrt);
    // Writing is performed
    signal(wrt);
}

// Reader
while (true) {
    wait(mutex);
    read_count++;
    signal(mutex);
    // Reading is performed
    wait(mutex);
    read_count--;
    signal(mutex);
}
```

- (a)** Correct/complete this solution.
- (b)** Explain how this solution satisfies the constraints of the Readers Writers Problem.
- (c)** Improve this code to guarantee bounded waiting for Writers.

10. Complete the following Monitor code for implementing a Bounded-Buffer satisfying its classical constraints. Explain your code.

```
monitor boundedBuffer {
    int items[MAX_ITEMS];
    int numItems = 0;

    condition full, empty;

    void produce(int v) {

    }

    int consume() {
        int retVal;

        return retVal;
    }
}
```

11. Consider the following attempt to generalize Peterson's Solution to three (3) processes to solve the Critical Section Problem (CSP). Assume that the processes are numbered 0, 1, and 2. Does this proposal meet the criteria of the CSP? Explain your answer.

```
void enterRegion(int process) {
    /* other1 and other2 are the IDs of the other two processes
    int other1 = (process + 1) % 3;
    int other2 = (process + 2) % 3;

    interested[process] = true;
    turn = other1;

    while (turn != process && (interested[other1] || interested[other2]))
        ; /* do nothing */
}

void leaveRegion(int process) {
    interested[process] = false;
}
```

12. The first known correct software solution to the critical-section problem for two processes was developed by *Dekker*. The two processes P_0 and P_1 share the following variables:

```
boolean flag[2]; /* initially false */
int turn;
```

The structure of process P_i ($i == 0$ or 1) is shown below. The other process is P_j ($j == 1$ or 0). Prove that the algorithm satisfies all three requirements for the Critical Section Problem.

```
do {
    flag[i] = true;
    while (flag[j]) {
        if (turn == j) {
            flag[i] = false;
            while (turn == j)
                ; /* do nothing */
            flag[i] = true;
        }
    }

    /* critical section */

    turn = j;
    flag[i] = false;

    /* remainder section */
}
while (true);
```


13. The first known correct software solution to the critical-section problem for n processes with a lower bound on waiting of $n - 1$ turns was presented by *Eisenberg* and *McGuire*. The processes share the following variables:

```
enum pstate {idle, want_in, incs};
pstate flag[n];
int turn;
```

All the elements of the flag are initially idle. The initial value of turn is immaterial (between 0 and $n - 1$). The structure of process P_i is shown below. Prove that the algorithm satisfies all three requirements for the Critical Section Problem.

```
do {
    while (true) {
        flag[i] = want_in;
        j = turn;

        while (j != i) {
            if (flag[j] != idle)
                j = turn;
            else
                j = (j + 1) % n;
        }

        flag[i] = in_cs;
        j = 0;

        while ( (j < n) && (j == i || flag[j] != in_cs))
            j++;

        if ( (j >= n) && (turn == i || flag[turn] == idle))
            break;
    }

    /* critical section */

    j = (turn + 1) % n;

    while (flag[j] == idle)
        j = (j + 1) % n;

    turn = j;
    flag[i] = idle;

    /* remainder section */
}
while (true);
```

14. Complete the following program to solve the requirements of the critical section problem. Then explain how each criterion is satisfied.

```
while (true) {
    waiting[i] = true;
    key = true;

    while (waiting[i] && key)
        key = Swap(&lock, &key);

    waiting[i] = false;

    /* Critical section */

    /* Remainder section */
}
```

15. Write a counting Semaphore implementation in pseudocode or C. Then improve your code to eliminate busy waiting.

16. Design a solution in pseudocode to solve the Critical Section Problem (CSP) using `test_and_set()` instruction. Prove that your solution satisfies all three criteria of the CSP solution.

17. Design a multithreaded Java or C program in which multiple threads access and update a shared variable. Show how race condition may occur and then provide a correct solution using semaphores. Explain how your solution satisfies the requirements of the Critical Section Problem

DEADLOCKS

1. Consider the following system resource-process state:

	<u>Allocation</u>				<u>Max</u>				<u>Available</u>			
	A	B	C	D	A	B	C	D	A	B	C	D
P_0	0	0	1	4	0	6	5	6	1	5	2	0
P_1	1	0	0	0	1	7	5	0				
P_2	1	3	5	4	2	3	5	6				
P_3	0	6	3	2	0	6	5	2				

- (a) Explain whether the system is in a safe state or is in a deadlock?
- (b) If P_0 requests for (1, 0, 0, 0), can the request be granted, why?
- (c) If P_2 requests for (1, 0, 0, 0), can the request be granted, why?
- (d) Find and explain the algorithmic time complexity of this deadlock detection system.

2. Consider the following system resource-process state:

	<u>Allocation</u>				<u>Max</u>				<u>Available</u>			
	A	B	C	D	A	B	C	D	A	B	C	D
P_0	0	0	1	2	0	0	1	3	1	5	2	0
P_1	1	0	0	0	1	7	5	0				
P_2	1	3	5	4	2	3	5	6				
P_3	0	6	3	2	0	6	5	2				
P_4	0	0	1	4	0	6	5	6				

- (a) Is the system in a safe state, why?
- (b) If P_2 requests for (1, 0, 1, 0), can the request be granted, why?
- (c) If P_2 requests for (1, 0, 0, 0), can the request be granted, why?
- (d) If a deadlock occurs in a system, what would be a good method to resolve it?

3. Consider the following system state when using Banker's Algorithm for deadlock avoidance.

	<u>Allocation</u>				<u>Max</u>				<u>Available</u>			
	A	B	C	D	A	B	C	D	A	B	C	D
P_0	0	0	1	2	0	0	1	3	1	5	2	0
P_1	1	0	0	0	1	7	5	0				
P_2	1	3	5	4	2	3	5	6				
P_3	0	6	3	2	0	6	5	2				
P_4	0	0	1	4	0	6	5	6				

- (a) What is the content of the matrix *Need*?
- (b) Is the system in a safe state, why?
- (c) If a request from process P_3 arrives for (0, 5, 2, 0), can the request be granted, why?
- (d) If a request from process P_1 arrives for (1, 5, 2, 0), can the request be granted, why?

4. Consider a system uses a deadlock detection algorithm:

	<u>Request</u>				<u>Allocation</u>				<u>Available</u>			
	A	B	C	D	A	B	C	D	A	B	C	D
P_0	0	0	1	2	0	0	1	3	1	5	2	0
P_1	1	0	0	0	1	7	5	0				
P_2	1	3	5	4	2	3	5	6				
P_3	0	6	3	2	0	6	5	2				
P_4	0	0	1	4	0	6	5	6				

- (a) Shortly define the deadlock detection system model.
- (b) Is the system in a safe state, why?
- (c) If P_1 requests for (0, 5, 2, 0), can the request be granted? Will the system be in deadlock, why?
- (d) If P_3 requests for (1, 5, 2, 0), can the request be granted? Will the system be in deadlock, why?
- (e) What is the algorithmic complexity of the deadlock detection procedure, why?

5. Consider the following system resource process state:

	<u>Allocation</u>				<u>Max</u>				<u>Available</u>			
	A	B	C	D	A	B	C	D	A	B	C	D
P_0	0	0	1	2	0	0	1	3	1	5	2	0
P_1	1	0	0	0	1	7	5	0				
P_2	1	3	5	4	2	3	5	6				
P_3	0	6	3	2	0	6	5	2				
P_4	0	0	1	4	0	6	5	6				

- (a) Is the system in a safe state, why?
- (b) If P_4 requests for (0, 5, 2, 0), can the request be granted immediately, why?
- (c) Assume the system uses deadlock detection algorithm and the request from process P_4 (0, 5, 2, 0) is granted. Will the system be in deadlock? If so, which processes will be in deadlock?
- (d) What can the system do if a deadlock is detected? What are the potential problems?

6. Consider the following snapshot of a system:

	<i>Allocation</i>	<i>Max</i>	<i>Available</i>
	<i>A B C D</i>	<i>A B C D</i>	<i>A B C D</i>
P_0	0 0 1 2	0 0 1 2	1 5 2 0
P_1	1 0 0 0	1 7 5 0	
P_2	1 3 5 4	2 3 5 6	
P_3	0 6 3 2	0 6 5 2	
P_4	0 0 1 4	0 6 5 6	

Answer the following questions using the banker's algorithm:

- (a) What is the content of the matrix *Need*?
- (b) Is the system in a safe state?
- (c) If a request from process P_1 arrives for (0, 4, 2, 0), can the request be granted immediately?

7. Consider the following snapshot of a system:

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	<i>A B C D</i>	<i>A B C D</i>	<i>A B C D</i>
P_0	2 0 0 1	4 2 1 2	3 3 2 1
P_1	3 1 2 1	5 2 5 2	
P_2	2 1 0 3	2 3 1 6	
P_3	1 3 1 2	1 4 2 4	
P_4	1 4 3 2	3 6 6 5	

Answer the following questions using the banker's algorithm:

- (a) Illustrate that the system is in a safe state by demonstrating an order in which the processes may complete.
- (b) If a request from process P_1 arrives for (1, 1, 0, 0), can the request be granted immediately?
- (c) If a request from process P_4 arrives for (0, 0, 2, 0), can the request be granted immediately?

MEMORY & VIRTUAL MEMORY

1. Consider a logical address space of 16 pages with 1024 bytes each, mapped onto a physical memory of 32 frames.

- (a)** How many bits are to be used for page number and offset in the logical address?
- (b)** How many bits are used in the physical address?

2. Describe a method to detect “*trashing*” with 8 reference bits in a virtual memory system.

3. For a virtual memory system, show the sequence of frames put into main memory with the following reference string using 4 available frames and LRU algorithm. What is the fault rate on this string?

4 8 2 7 4 5 9 1 8 7 4 5 2 7

4. For a virtual memory system, show the sequence of frames put into main memory with the following reference string:

5 4 8 2 7 4 5 9 1 8 7 4 5 2 7

Which gives the lowest fault rate on this string?

- (a) Using 4 available frames and the “LRU (Least-Recently Used)” algorithm.
- (b) Using 4 available frames and the “Second Change” algorithm

5. Consider a virtual memory system with a physical memory of 4 frames. With the reference string:

7, 0, 1, 2, 0, 3, 4, 0.

Show the state of the frames step by step

- (a) using the FIFO (First-in First-out) algorithm, then find the number of page faults and hits.
- (b) using the LRU (Least Recently Used) algorithm, then find the number of page faults and hits.
- (c) using the Additional Reference Bits algorithm using 8-bit registers, then find the number of page faults and hits.